

Завдання 1. Розробити Python-скрипт на базі бібліотеки *boto3*, який матиме наступний функціонал (параметри передаються як аргументи)

a. EC2

i. Створити новий інстанс з вказаним типом та шаблоном (з тих, які відповідають *Free Tier*)

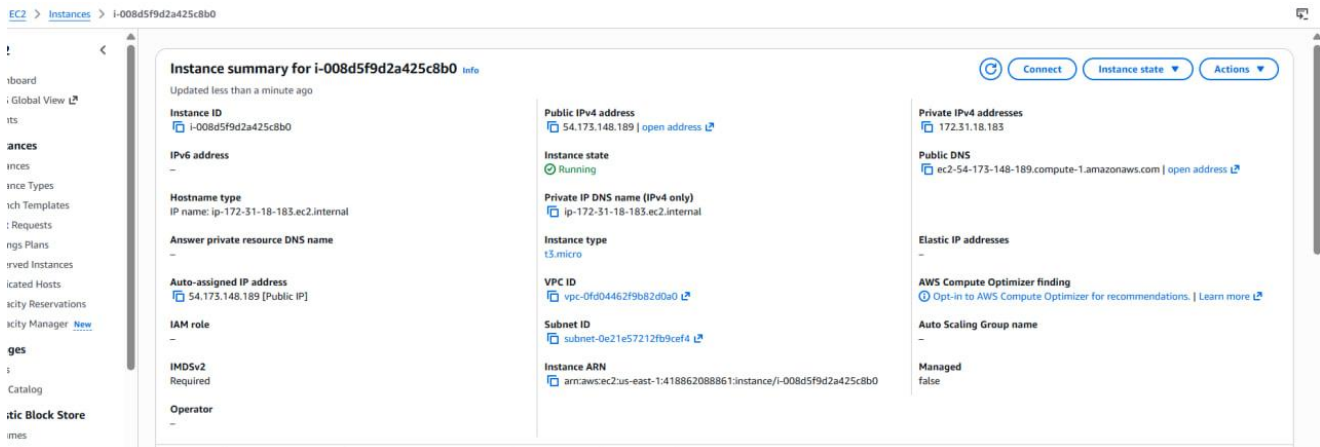
У цьому фрагменті я передаю регіон, ідентифікатор шаблону, тип інстансу, назву ключа та файл user data як аргументи. Далі за допомогою `run_instances` створюється один інстанс, після чого програма чекає, поки він перейде у стан `running`

```
def ec2_create(a):
    region=need(a,"region")
    template_id=need(a,"template_id")
    instance_type=need(a,"instance_type")
    key_name=need(a,"key_name")
    user_file=need(a,"user_data")
    ec2=boto3.client("ec2",region_name=region)
    with open(user_file,"r",encoding="utf-8") as f:
        user_data=f.read()
    res=ec2.run_instances(
        LaunchTemplate={"LaunchTemplateId":template_id},
        InstanceType=instance_type,
        KeyName=key_name,
        UserData=user_data,
        MinCount=1,
        MaxCount=1
    )
    instance_id=res["Instances"][0]["InstanceId"]
    print("Instance created:",instance_id)
    ec2.get_waiter("instance_running").wait(InstanceIds=[instance_id])
    print("Instance is running")
```

Виконання команди для створення інстансу та результат:

```
PS S:\навчання\cloudtech\лаб4> py lab4.py ec2-create region=us-east-1 template_id=lt-01d6a92b25450acfd instance_type=t3.micro key_name=pair1 user_data=setup.sh
Instance created: i-008d5f9d2a425c8b0
Instance is running
PS S:\навчання\cloudtech\лаб4> █
```

Створений новий інстанс в AWS:



*ii. Реалізувати можливість при створенні інстансу передавання **bash** скрипту, який поставить додаткові бібліотеки під час створення інстансу (наприклад **rpm** та кілька пакетів, встановлених з його виходу ристанням на ваш вибір)*

Bash-скрипт, який передавався при створенні інстансу:

```
#!/bin/bash
sudo yum update -y
sudo yum install -y httpd
sudo systemctl start httpd
sudo systemctl enable httpd
echo "Hello from lab 4" | sudo tee /var/www/html/index.html
```

Bash-скрипт передається під час створення інстансу через параметр UserData:

```
def ec2_create(a):
    region=need(a,"region")
    template_id=need(a,"template_id")
    instance_type=need(a,"instance_type")
    key_name=need(a,"key_name")
    user_file=need(a,"user_data")
    ec2=boto3.client("ec2",region_name=region)
    with open(user_file,"r",encoding="utf-8") as f:
        user_data=f.read()
    res=ec2.run_instances(
        LaunchTemplate={"LaunchTemplateId":template_id},
        InstanceType=instance_type,
        KeyName=key_name,
        UserData=user_data,
        MinCount=1,
        MaxCount=1
```

У логах видно виконання команд із bash-скрипту, зокрема встановлення пакета httpd та виведення повідомлення

```
Installed:
apr-1.7.5-1.amzn2023.0.4.x86_64
apr-util-1.6.3-1.amzn2023.0.2.x86_64
apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64
apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
httpd-2.4.66-1.amzn2023.0.1.x86_64
httpd-core-2.4.66-1.amzn2023.0.1.x86_64
httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch
httpd-tools-2.4.66-1.amzn2023.0.1.x86_64
libbrotli-1.0.9-4.amzn2023.0.2.x86_64
mailcap-2.1.49-3.amzn2023.0.3.noarch
mod_http2-2.0.27-1.amzn2023.0.3.x86_64
mod_lua-2.4.66-1.amzn2023.0.1.x86_64

Complete!
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service.
Hello from lab 4
cloud-init v. 22.2.2 finished at Tue, 10 Mar 2026 23:17:49 +0000
```

iii. Змінити групу безпеки інстансу, додавши правила для групи без неки (22 та 80 порт для вхідного трафіку)

За допомогою функції `authorize_security_group_ingress` додаю правила для вхідного трафіку.

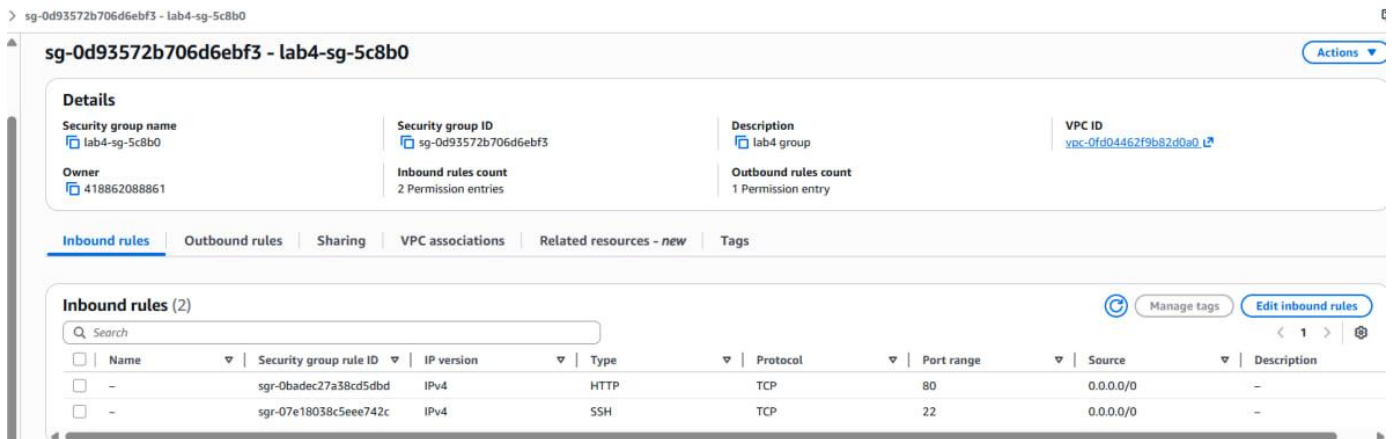
У коді відкриваються два порти: 22 SSH для підключення до інстансу та 80 HTTP для доступу до веб-сервера, доступ дозволяється з будь-якої IP-адреси

```
def ec2_ports(a):
    region=need(a,"region")
    instance_id=need(a,"instance_id")
    ec2=boto3.client("ec2",region_name=region)
    res=ec2.describe_instances(InstanceIds=[instance_id])
    inst=res["Reservations"][0]["Instances"][0]
    vpc_id=inst["VpcId"]
    sg_name="lab4-sg-"+instance_id[-5:]
    sg=ec2.create_security_group(Group_name=sg_name,Description="lab4 group",VpcId=vpc_id)
    sg_id=sg["GroupId"]
    ec2.authorize_security_group_ingress(
        GroupId=sg_id,
        IpPermissions=[
            {"IpProtocol":"tcp","FromPort":22,"ToPort":22,"IpRanges":[{"CidrIp":"0.0.0.0/0"}]},
            {"IpProtocol":"tcp","FromPort":80,"ToPort":80,"IpRanges":[{"CidrIp":"0.0.0.0/0"}]}
        ]
    )
    groups=[]
    for x in inst.get("SecurityGroups",[]):
        groups.append(x["GroupId"])
    groups.append(sg_id)
    ec2.modify_instance_attribute(InstanceId=instance_id,Groups=groups)
    print("Ports 22 and 80 opened")
    print("Security group:",sg_id)
```

Виконання команди для зміни правил групи безпеки:

```
PS S:\навчання\cloudtech\лаб4> py lab4.py ec2-ports region=us-east-1 instance_id=i-008d5f9d2a425c8b0
Ports 22 and 80 opened
Security group: sg-0d93572b706d6ebf3
```

У правилах Inbound rules видно, що відкриті порти 22 та 80



iv. Для існуючого інстансу змінити назву та додати описові теги

Спочатку передаються параметри регіон, ID інстансу, нова назва та додаткові теги.

Після цього формується список тегів і за допомогою функції `create_tags` і вони додаються до інстансу

```
def ec2_tags(a):
    region=need(a,"region")
    instance_id=need(a,"instance_id")
    name=need(a,"name")
    ec2=boto3.client("ec2",region_name=region)
    tags=[{"Key":"Name","Value":name}]
    if "tags" in a and a["tags"]!="":
        for x in a["tags"].split(";"):
            if "=" in x:
                k,v=x.split("=",1)
                tags.append({"Key":k,"Value":v})
    ec2.create_tags(Resources=[instance_id],Tags=tags)
    print("Tags updated")
```

Виконання команди для зміни назви та додавання тегів:

```
PS S:\навчання\cloudtech\лаб4> py lab4.py ec2-tags region=us-east-1 instance_id=i-008d5f9d2a425c8b0 name=lab4-instance "tags=Owner=sonkka;Lab=4"
Tags updated
PS S:\навчання\cloudtech\лаб4>
```

Перевіряю зміну у AWS, бачимо що з'явилась назва та додалися теги:

Details	Status and alarms	Monitoring	Security	Networking	Storage	Tags
Tags						
Key	Value					
Lab	4					
Owner	sonkka					
aws:ec2launc...	1					
aws:ec2launc...	lt-01d6a92b25450acfd					
Name	lab4-instance					

v. Отримання детальної описової інформації щодо інстансу за його ідентифікатором (RAM, Диск, тип, ір-адреса)

Спочатку скрипт звертається до AWS і знаходить потрібний інстанс далі окремо визначаються його тип, публічна IP-адреса, обсяг оперативної пам'яті та розмір диска та після цього вся основна інформація виводиться в консоль

```
def ec2_info(a):
    region=need(a,"region")
    instance_id=need(a,"instance_id")
    ec2=boto3.client("ec2",region_name=region)
    res=ec2.describe_instances(InstanceIds=[instance_id])
    inst=res["Reservations"][0]["Instances"][0]
    instance_type=inst["InstanceType"]
    ip=inst.get("PublicIpAddress","No public IP")
    t=ec2.describe_instance_types(InstanceTypes=[instance_type])["InstanceTypes"][0]
    ram=t["MemoryInfo"]["SizeInMiB"]
    vol_ids=[]
    for x in inst.get("BlockDeviceMappings",[]):
        if "Ebs" in x and "VolumeId" in x["Ebs"]:
            vol_ids.append(x["Ebs"]["VolumeId"])
    disk=0
    if vol_ids:
        vols=ec2.describe_volumes(VolumeIds=vol_ids)["Volumes"]
        for v in vols:
            disk+=v["Size"]
    print("Instance ID:",instance_id)
    print("Type:",instance_type)
    print("RAM:",ram,"MiB")
    print("Disk:",disk,"GB")
    print("IP:",ip)
```

Команда, бачимо всю потрібну інформацію:

```
PS S:\навчання\cloudtech\ла64> py lab4.py ec2-info region=us-east-1 instance_id=i-008d5f9d2a425c8b0
Instance ID: i-008d5f9d2a425c8b0
Type: t3.micro
RAM: 1024 MiB
Disk: 8 GB
IP: 54.173.148.189
PS S:\навчання\cloudtech\ла64> \
```

vi. Зупинка, запуск та видалення існуючого інстансу за ідентифікатором

Для зупинки використовується метод `stop_instances`, для повторного запуску `start_instances`, а для видалення `terminate_instances`, у кожному випадку я передаю регіон та `instance_id` як аргументи командного рядка

```
def ec2_stop(a):
    region=need(a,"region")
    instance_id=need(a,"instance_id")
    boto3.client("ec2",region_name=region).stop_instances(InstanceIds=[instance_id])
    print("Stop started")

def ec2_start(a):
    region=need(a,"region")
    instance_id=need(a,"instance_id")
    boto3.client("ec2",region_name=region).start_instances(InstanceIds=[instance_id])
    print("Start started")

def ec2_delete(a):
    region=need(a,"region")
    instance_id=need(a,"instance_id")
    boto3.client("ec2",region_name=region).terminate_instances(InstanceIds=[instance_id])
    print("Delete started")
```

Виконання команди для зупинки існуючого інстансу:

```
python lab4.py aws-delete-table region=... table=...
PS S:\навчання\cloudtech\лаб4> py lab4.py ec2-stop region=us-east-1 instance_id=i-008d5f9d2a425c8b0
Stop started
PS S:\навчання\cloudtech\лаб4>
```

В AWS бачимо що інстанс перейшов у стан stopped

Instances (2) Info				
Find Instance by attribute or tag (case-sensitive)				
☐	Name ✎	Instance ID	Instance state ▼	Instance type ▼
☐	labka1	i-070346724e9ef7da7	✔ Running 🔍 🔍	t3.micro
☐	lab4-instance	i-008d5f9d2a425c8b0	⊖ Stopped 🔍 🔍	t3.micro

Виконання команди для повторного запуску інстансу після зупинки:

```
python lab4.py aws-delete-table region=... table=...
PS S:\навчання\cloudtech\лаб4> py lab4.py ec2-start region=us-east-1 instance_id=i-008d5f9d2a425c8b0
Start started
PS S:\навчання\cloudtech\лаб4>
```

В AWS бачимо що інстанс знову у стані Running

Instances (2) [Info](#)

Find Instance by attribute or tag (case-sensitive)

☐	Name ✎	Instance ID	Instance state ▼
☐	labka1	i-070346724e9ef7da7	Running ⚙ 🔍
☐	lab4-instance	i-008d5f9d2a425c8b0	Running ⚙ 🔍

Виконання команди для видалення існуючого інстансу

```
PS S:\навчання\cloudtech\лаб4> py lab4.py ec2-delete region=us-east-1 instance_id=i-008d5f9d2a425c8b0
Delete started
PS S:\навчання\cloudtech\лаб4>
```

Інстанс був переведений у стан terminated

Instances (2) [Info](#)

Find Instance by attribute or tag (case-sensitive)

☐	Name ✎	Instance ID	Instance state ▼
☐	labka1	i-070346724e9ef7da7	Running ⚙ 🔍
☐	lab4-instance ✎	i-008d5f9d2a425c8b0	Terminated ⚙ 🔍

b. S3

i. Створення та видалення бакету з певною назвою на регіоні, що передаються як параметри

Назва бакета і регіон передаються як аргументи, тому код не треба змінювати вручну під кожен новий бакет. Для створення використовується метод `create_bucket`, а для видалення `delete_bucket`. Якщо регіон не `us-east-1`, додатково передається `LocationConstraint`

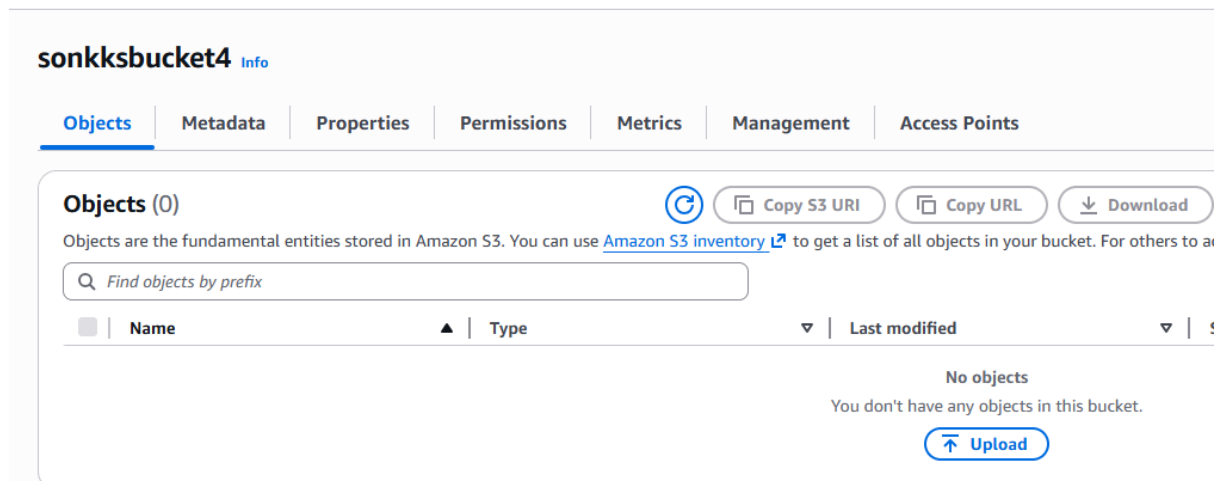
```
def s3_create_bucket(a):
    region=need(a,"region")
    bucket=need(a,"bucket")
    s3=boto3.client("s3",region_name=region)
    if region=="us-east-1":
        s3.create_bucket(Bucket=bucket)
    else:
        s3.create_bucket(Bucket=bucket,CreateBucketConfiguration={"LocationConstraint":region})
    print("Bucket created:",bucket)
```

```
def s3_delete_bucket(a):
    region=need(a,"region")
    bucket=need(a,"bucket")
    s3=boto3.client("s3",region_name=region)
    s3.delete_bucket(Bucket=bucket)
    print("Bucket deleted")
```

Виконання команди для створення нового бакета з указаної назвою та регіоном:

```
PS S:\навчання\cloudtech\лаб4> py lab4.py s3-create-bucket region=us-east-1 bucket=sonkksbucket4
Bucket created: sonkksbucket4
```

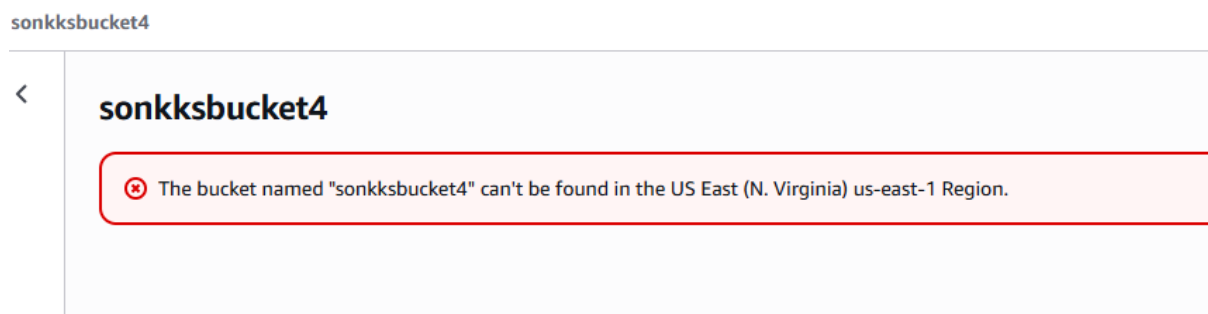
Бачимо створений бакет у консолі AWS



Виконання команди для видалення бакета:

```
PS S:\навчання\cloudtech\лаб4> py lab4.py s3-delete-bucket region=us-east-1 bucket=sonkksbucket4
Bucket deleted
```

Після оновлення сторінки бакета бачимо помилку, бакета більше не існує



ii. Завантаження файлу на бакет (в тому числі той файл, з якого брались дані у лабі 3 для наповнення DупатоDB таблиці)

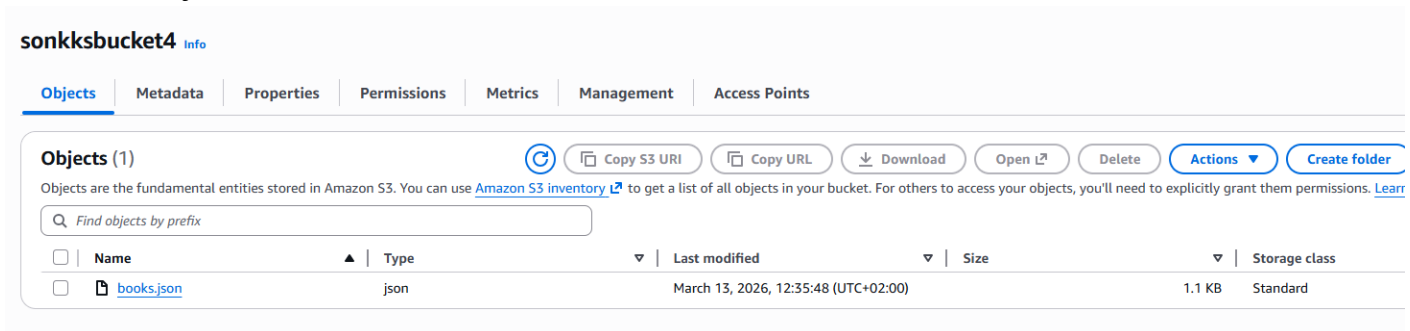
Як аргументи передаються регіон, назва бакета, шлях до локального файлу та ім'я файлу в бакеті, скрипт використовує функцію upload_file

```
def s3_upload_file(a):
    region=need(a,"region")
    bucket=need(a,"bucket")
    file_path=need(a,"file")
    key=a["key"] if "key" in a else os.path.basename(file_path)
    boto3.client("s3",region_name=region).upload_file(file_path,bucket,key)
    print("File uploaded:",key)
```

Виконання команди для завантаження файлу у бакет. У моєму випадку я завантажила файл books.json, який використовувався в попередній лабораторній роботі

```
PS S:\навчання\cloudtech\лаб4> py lab4.py s3-upload-file region=us-east-1 bucket=sonkksbucket4 "file=S:\навчання\cloudtech\лаб3\books.json" key=books.json
File uploaded: books.json
PS S:\навчання\cloudtech\лаб4>
```

Бачимо вміст бакета після завантаження файлу та у списку об'єктів з'явився файл books.json



iii. Завантаження папки з файлами на бакет

Скрипт проходить по всіх файлах у вказаній папці за допомогою os.walk.

Кожен знайдений файл окремо завантажується у бакет через функцію upload_file.

Таким чином можна автоматично завантажити одразу багато файлів без ручного додавання кожного

```
def s3_upload_folder(a):
    region=need(a,"region")
    bucket=need(a,"bucket")
    folder=need(a,"folder")
    prefix=a.get("prefix","")
    s3=boto3.client("s3",region_name=region)
    for root,dirs,files in os.walk(folder):
        for file_name in files:
            full_path=os.path.join(root,file_name)
            rel_path=os.path.relpath(full_path,folder).replace("\\","/")
            if prefix:
                key=prefix.rstrip("/")+"/"+rel_path
            else:
                key=rel_path
            s3.upload_file(full_path,bucket,key)
            print("Uploaded:",key)
```

Виконання команди для завантаження папки з файлами у S3-бакет, у консолі видно список файлів, які були успішно завантажені:

```
PS S:\навчання\cloudtech\лаб4> py lab4.py s3-upload-folder region=us-east-1 bucket=sonkksbucket4 "folder=S:\навчання\cloudtech\test"
Uploaded: Лист Microsoft Excel.xlsx
Uploaded: Презентация Microsoft PowerPoint.pptx
Uploaded: Текстовий документ.txt
PS S:\навчання\cloudtech\лаб4>
```

Бачимо вміст бакета sonkksbucket4 після виконання скрипту і у списку об'єктів видно файли, які були завантажені з локальної папки

sonkksbucket4 [Info](#)

Objects Metadata Properties Permissions Metrics Management Access Points

Objects (4)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions.

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	Лист Microsoft Excel.xlsx	xlsx	March 13, 2026, 12:40:35 (UTC+02:00)	6.0 KB	Standard
☐	Презентация Microsoft PowerPoint.pptx	pptx	March 13, 2026, 12:40:35 (UTC+02:00)	0 B	Standard
☐	Текстовий документ.txt	txt	March 13, 2026, 12:40:35 (UTC+02:00)	0 B	Standard
☐	books.json	json	March 13, 2026, 12:35:48 (UTC+02:00)	1.1 KB	Standard

iv. Отримання лістингу файлів (рекурсивно для папок) на бакеті з назвами, класом зберігання, версіями, розміром

Скрипт проходить по всіх об'єктах у бакеті за допомогою пагінатора, тому може працювати і з вкладеними папками і для кожного файлу виводиться його назва, клас зберігання, версії та розмір, якщо бакет порожній, скрипт окремо виводить повідомлення про це

```
def s3_list(a):
    region=need(a,"region")
    bucket=need(a,"bucket")
    s3=boto3.client("s3",region_name=region)
    ver_map={}
    vers=s3.list_object_versions(Bucket=bucket)
    for x in vers.get("Versions",[]):
        if x["Key"] not in ver_map:
            ver_map[x["Key"]]=[]
            ver_map[x["Key"]].append(x["VersionId"])
    empty=True
    for page in s3.get_paginator("list_objects_v2").paginate(Bucket=bucket):
        for obj in page.get("Contents",[]):
            empty=False
            key=obj["Key"]
            print("Name:",key)
            print("Storage class:",obj.get("StorageClass","STANDARD"))
            print("Versions:",ver_map.get(key,"no versions"))
            print("Size:",obj["Size"])
            print("----")
    if empty:
        print("Bucket is empty")
```

Виконання команди для отримання лістингу файлів у S3-бакеті у консолі видно назви файлів, клас зберігання, версії та розмір кожного об'єкта:

```

PS S:\навчання\cloudtech\лаб4> py lab4.py s3-list region=us-east-1 bucket=sonkksbucket4
Name: books.json
Storage class: STANDARD
Versions: ['null']
Size: 1111
---
Name: Лист Microsoft Excel.xlsx
Storage class: STANDARD
Versions: ['null']
Size: 6186
---
Name: Презентация Microsoft PowerPoint.pptx
Storage class: STANDARD
Versions: ['null']
Size: 0
---
Name: Текстовий документ.txt
Storage class: STANDARD
Versions: ['null']
Size: 0

```

v. Зробити файл на бакеті публічним

Скрипт отримує назву бакета та ключ файлу, після чого за допомогою `put_object_acl` встановлює для нього права `public-read`. У результаті в консоль виводиться пряме посилання

```

def s3_public(a):
    region=need(a,"region")
    bucket=need(a,"bucket")
    key=need(a,"key")
    s3=boto3.client("s3",region_name=region)
    s3.put_object_acl(Bucket=bucket,Key=key,ACL="public-read")
    print("File is public")
    print(f"https://{bucket}.s3.amazonaws.com/{key}")

```

Виконання команди для надання файлу `books.json` публічного доступу в бакеті:

```

PS S:\навчання\cloudtech\лаб4> py lab4.py s3-public region=us-east-1 bucket=sonkksbucket4 key=books.json
File is public
https://sonkksbucket4.s3.amazonaws.com/books.json

```

Після зміни прав доступу файл став публічним

books.json [Info](#)

[Copy S3 URI](#)
[Download](#)
[Open](#)
[Object actions](#)

[Properties](#)
[Permissions](#)
[Versions](#)

Access control list (ACL)

Grant basic read/write permissions to AWS accounts. [Learn more](#)

Grantee

Object

Object ACL

Object owner (your AWS account)

Canonical ID: [de7a2993a8431d3ee926ceb35597c853469230d53f9cdf5a4ce5bc4df474b010](#)

Read

Read, Write

Everyone (public access)

Group: [http://acs.amazonaws.com/groups/global/AllUsers](#)

Read

-

Authenticated users group (anyone with an AWS account)

Group: [http://acs.amazonaws.com/groups/global/AuthenticatedUsers](#)

-

-

vi. Видалити файли чи папки на бакеті

Спочатку програма отримує регіон, назву бакета та шлях до об'єкта або папки, далі за допомогою `list_objects_v2` виконується пошук усіх об'єктів, які мають вказаний префікс. Якщо за шляхом нічого не знайдено, програма виводить повідомлення `Nothing found`. Якщо об'єкти знайдено, вони видаляються методом `delete_objects`, після чого виводиться повідомлення `Deleted`

```
def s3_delete_path(a):
    region=need(a,"region")
    bucket=need(a,"bucket")
    path=need(a,"path")
    s3=boto3.client("s3",region_name=region)
    objs=[]
    for page in s3.get_paginator("list_objects_v2").paginate(Bucket=bucket,Prefix=path):
        for obj in page.get("Contents",[]):
            objs.append({"Key":obj["Key"]})
    if not objs:
        print("Nothing found")
        return
    s3.delete_objects(Bucket=bucket,Delete={"Objects":objs})
    print("Deleted")
```

Виконання команди для видалення файлу:

```
PS S:\навчання\cloudtech\лаб4> py lab4.py s3-delete-path region=eu-north-1 bucket=sonkksbucket4 path=Лист Microsoft Excel.xlsx
Deleted
PS S:\навчання\cloudtech\лаб4>
```

Бачимо що цей файл відсутній

sonkksbucket4 [Info](#)

[Objects](#)
[Metadata](#)
[Properties](#)
[Permissions](#)
[Metrics](#)
[Management](#)

Objects (3)

Copy S3 U

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get

☐	Name	Type
☐	Презентация Microsoft PowerPoint.pptx	pptx
☐	Текстовий документ.txt	txt
☐	books.json	json

vii. Видалити бакет

```
def s3_delete_bucket(a):
    region=need(a,"region")
    bucket=need(a,"bucket")
    s3=boto3.client("s3",region_name=region)
    s3.delete_bucket(Bucket=bucket)
    print("Bucket deleted")
```

Виконання команди для видалення бакета:

```
PS S:\навчання\cloudtech\лаб4> py lab4.py s3-delete-bucket region=us-east-1 bucket=sonkksbucket4
Bucket deleted
```

Після оновлення сторінки бакета бачимо помилку, бакета більше не існує

sonkksbucket4

<

sonkksbucket4

The bucket named "sonkksbucket4" can't be found in the US East (N. Virginia) us-east-1 Region.

c. DynamoDB

i. Створити таблицю з певною назвою

через `boto3.resource` створюю таблицю за допомогою `create_table`. У параметрах задається ім'я таблиці, первинний ключ типу HASH, опис атрибута ключа та режим оплати `PAY_PER_REQUEST`, при якому оплата здійснюється за фактичне

використання. Після цього програма очікує, поки таблиця буде повністю створена, і виводить повідомлення Table created

```
def ddb_create_table(a):
    region=need(a,"region")
    table_name=need(a,"table")
    key_name=need(a,"key_name")
    key_type=a.get("key_type","S")
    ddb=boto3.resource("dynamodb",region_name=region)
    table=ddb.create_table(
        TableName=table_name,
        KeySchema=[{"AttributeName":key_name,"KeyType":"HASH"}],
        AttributeDefinitions=[{"AttributeName":key_name,"AttributeType":key_type}],
        BillingMode="PAY_PER_REQUEST"
    )
    table.wait_until_exists()
    print("Table created")
```

Виконання команди для створення таблиці в DynamoDB:

```
PS S:\навчання\cloudtech\лаб4> py lab4.py ddb-create-table region=us-east-1 table=books-table key_name=id key_type=S
Table created
PS S:\навчання\cloudtech\лаб4>
```

Бачимо створену таблицю у консолі AWS

The screenshot shows the AWS Management Console interface for a DynamoDB table named 'books-table'. On the left, a sidebar lists four tables: 'books-table' (selected), 'sonkkstable', 'sonkkstable2', and 'sonkkstable3'. The main panel displays the 'books-table' details under the 'Settings' tab. At the top, it shows the table name, last updated time (March 13, 2026, 13:25 UTC+2:00), and buttons for 'Actions' and 'Explore table items'. Below this is a warning banner about protecting the table from accidental writes and deletes using PITR. The 'General information' section includes: Partition key (id (String)), Sort key (-), Capacity mode (On-demand), Alarms (No active alarms), Point-in-time recovery (PITR) (Off), Resource-based policy (Not active), Average item size (0 bytes), Item count (0), Table status (Active), and Table size (0 bytes). The Amazon Resource Name (ARN) is also displayed.

ii. Наповнити її даними з файлу на S3

Ця частина коду зчитує вміст файлу та визначає його формат за розширенням. Якщо це CSV, дані обробляються через csv.DictReader, якщо JSON то через json.loads. Далі записи перетворюються у потрібний формат і додаються в таблицю за допомогою batch_writer

```

def ddb_load_from_s3(a):
    region=need(a,"region")
    table_name=need(a,"table")
    bucket=need(a,"bucket")
    key=need(a,"key")
    s3=boto3.client("s3",region_name=region)
    ddb=boto3.resource("dynamodb",region_name=region)
    table=ddb.Table(table_name)
    text=s3.get_object(Bucket=bucket,Key=key)["Body"].read().decode("utf-8")
    rows=[]
    def convert_row(row):
        new_row={}
        for k,v in row.items():
            if k is None or v is None:
                continue
            try:
                new_row[k]=Decimal(str(v))
            except:
                new_row[k]=v
        return new_row
    if key.lower().endswith(".csv"):
        for row in csv.DictReader(io.StringIO(text)):
            new_row=convert_row(row)
            if new_row:
                rows.append(new_row)
    elif key.lower().endswith(".json"):
        data=json.loads(text)
        if isinstance(data, list):
            for row in data:
                if isinstance(row, dict):
                    new_row=convert_row(row)
                    if new_row:
                        rows.append(new_row)
        elif isinstance(data, dict):
            new_row=convert_row(data)
            if new_row:
                rows.append(new_row)
        else:
            print("Unsupported file format")
            return
    with table.batch_writer() as batch:

```

Виконання команди для завантаження даних із файлу, розміщеного в бакеті, до таблиці DynamoDB:

```

PS S:\навчання\cloudtech\лаб4> py lab4.py ddb-load-from-s3 region=us-east-1 table=books-table bucket=sonkksbucket4 key=books.json
Rows added: 10
PS S:\навчання\cloudtech\лаб4>

```

Бачимо записи, що були успішно завантажені до таблиці DynamoDB з файлу, розміщеного в S3

Table: books-table - Items returned (10)

Scan started on March 13, 2026, 13:35:12

☐	id (String)	author	genre	rating	title	year
☐	b06	Patrick Rot...	fantasy	9	The Name ...	2007
☐	b09	Stieg Larsson	mystery	8	The Girl wit...	2005
☐	b10	Liane Moria...	mystery	9	Big Little Lies	2014
☐	b04	Sally Thorne	romance	7	The Hating ...	2016
☐	b08	Aldous Huxl...	dystopia	8	Brave New ...	1932
☐	b03	Gillian Flynn	thriller	8	Gone Girl	2012
☐	b02	Donna Tartt	dark_acade...	10	The Secret ...	1992
☐	b07	George Or...	dystopia	10	1984	1949
☐	b01	Leigh Bardu...	dark_acade...	9	Ninth House	2019
☐	b05	Brandon Sa...	fantasy	9	Mistborn	2006

iii. Отримати дані з таблиці

Програма підключається до таблиці, виконує метод scan та отримує всі записи. Після цього дані виводяться у форматі JSON

```
def ddb_scan(a):  
    region=need(a,"region")  
    table_name=need(a,"table")  
    ddb=boto3.resource("dynamodb",region_name=region)  
    items=ddb.Table(table_name).scan().get("Items",[])  
    print(json.dumps(items,ensure_ascii=False,indent=2,default=str))
```

Виконання команди для отримання даних із таблиці:

```

PS S:\навчання\cloudtech\лаб4> py lab4.py ddb-scan region=us-east-1 table=books-table
[
  {
    "year": "2007",
    "id": "b06",
    "genre": "fantasy",
    "rating": "9",
    "title": "The Name of the Wind",
    "author": "Patrick Rothfuss"
  },
  {
    "year": "2005",
    "id": "b09",
    "genre": "mystery",
    "rating": "8",
    "title": "The Girl with the Dragon Tattoo",
    "author": "Stieg Larsson"
  },
  {
    "year": "2014",
    "id": "b10",
    "genre": "mystery",
    "rating": "9",
    "title": "Big Little Lies",
    "author": "Liane Moriarty"
  },
  {
    "year": "2016",
    "id": "b04",
    "genre": "romance",
    "rating": "7",
    "title": "The Hating Game",
    "author": "Sally Thorne"
  },
  {
    "year": "1932",
    "id": "b08",
    "genre": "dystopia",
    "rating": "8",

```

iv. Видалити таблицю

Програма отримує регіон і назву таблиці, після чого через клієнт boto3 викликає метод delete_table. Після успішного виконання виводиться повідомлення Table deleted.

```

def ddb_delete_table(a):
    region=need(a,"region")
    table_name=need(a,"table")
    boto3.client("dynamodb",region_name=region).delete_table(TableName=table_name)
    print("Table deleted")

```

Виконання команди для видалення таблиці:

```

PS S:\навчання\cloudtech\лаб4> py lab4.py ddb-delete-table region=us-east-1 table=books-table
Table deleted

```

Бачимо помилку при оновленні сторінки з таблицею, у списку таблиць ця таблиця відсутня

Tables (3) <

Filter by tag
Any tag key ▾

Filter by tag value
Any tag value ▾

🔍 Find tables

< 1 > ⚙️

☐ sonkkstable ☆

☐ sonkkstable2 ☆

☐ sonkkstable3 ☆

books-table

❌ Requested resource not found: Table: books-table not found

Висновок

У цій лабораторній роботі я на практиці розібралась, як працювати з сервісами AWS через Python. Я створювала і налаштовувала EC2, працювала з S3-бакетом, а також створювала, заповнювала і видаляла таблицю в DynamoDB. Під час виконання роботи я краще зрозуміла, як автоматизувати такі дії за допомогою boto3, а не робити все вручну через консоль.